

**SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY**



(SUBJECT: ARTIFICIAL NEURAL NETWORK)

(SUBJECTCODE:24SBT113)

A Individual Task on

**“Error-Correction Learning and Convergence Analysis
of Perceptron on AND/OR Tasks”**

*Submitted in the partial fulfillment of the requirements for the fourth
semester*

**BACHELOR OF
TECHNOLOGY IN AIML**

Submitted By,

SAMARTH SATISH NAIK (01SU24AI090)

UNDER THE GUIDANCE OF

Prof. Mahesh Kumar V B

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

**SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146**

2025-26

SRINIVAS UNIVERSITY

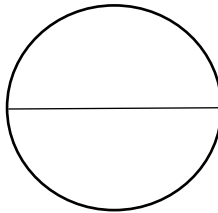
INSTITUTE OF ENGINEERING & TECHNOLOGY



Department of ARTIFICIAL INTELLIGENCE & MACHINE LEARNING CERTIFICATE

This is to certify that SAMARTH SATISH NAIK (01SU24AI090) has satisfactorily completed the assessment (Individual-Task – Module 3) in “**ARTIFICIAL NEURAL NETWORK** ” prescribed by the Srinivas University for the 4st semester B. Tech course during the year **2025-26**.

MARKS AWARDED



Staff In charge

Name: Prof. Mahesh Kumar V B

Assistant Professor, Department Of AIML

TABLE OF CONTENTS

Sl.no	Description	Page no.
1	<u>Introduction</u>	1
2	<u>Objective</u>	2
3	<u>Error-Correction Learning Rule</u>	3
4	<u>Problem Statement</u>	4
5	<u>Dataset Description</u>	5-6
6	<u>Algorithm</u>	7-8
7	<u>Manual Training Demonstration</u>	8-10
8	<u>Python code Implementation</u>	11-12
9	<u>Error Plot and Convergence Analysis</u>	13-14
10	<u>Effect of Learning Rate on Convergence</u>	15
11	<u>Conclusion</u>	16
12	<u>References</u>	17

1. Introduction to Artificial Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functioning of the human brain. They are designed to simulate the way biological neurons process information, enabling machines to learn from data and make intelligent decisions. The fundamental idea behind ANN is to create interconnected processing units, called artificial neurons, that work together to solve complex problems such as classification, prediction, pattern recognition, and decision-making.

The concept of neural networks originated from research in neuroscience and was mathematically modelled in the early twentieth century. One of the earliest artificial neuron models was proposed by Warren McCulloch and Walter Pitts, who demonstrated how biological neurons could be represented using logical functions. Later, the development of the Perceptron by Frank Rosenblatt provided a practical learning algorithm that allowed neural networks to adjust their parameters automatically based on errors. An artificial neural network consists of multiple layers of neurons arranged in three main types: the input layer, hidden layer(s), and output layer. The input layer receives data from the external environment. The hidden layers process this information through weighted connections, and the output layer produces the final result. Each neuron computes a weighted sum of its inputs, adds a bias value, and applies an activation function to produce an output. The activation function introduces non-linearity, allowing the network to solve complex real-world problems.

Learning in ANN occurs through the adjustment of weights and biases. During training, the network compares its predicted output with the actual target value and calculates an error. Based on this error, a learning rule is applied to modify the weights so that future predictions become more accurate. Common learning approaches include supervised learning, unsupervised learning, and reinforcement learning. In supervised learning, labelled data is used to guide the training process, making it one of the most widely applied methods in practical applications.

Artificial neural networks have gained significant importance due to their ability to handle large datasets and recognize intricate patterns that traditional algorithms may fail to detect. They are widely used in areas such as image recognition, speech processing, medical diagnosis, financial forecasting, natural language processing, and autonomous systems. With the advancement of computational power and availability of large-scale data, ANN has become a core component of modern artificial intelligence systems.

2.Objective of the Study

The primary objective of this study is to understand and model Artificial Neural Networks (ANN) for solving a real-life problem by mapping its structure, neuron types, and suitable learning law. The study aims to bridge theoretical knowledge of neural networks with practical implementation by designing an ANN architecture tailored to a selected application. Through this process, the working principles of neural networks, including forward propagation, weight adjustment, and learning mechanisms, are examined in detail.

Another important objective is to analyse how different layers in an ANN contribute to problem-solving. The input layer is responsible for receiving real-world data in numerical form, the hidden layers perform intermediate computations and feature extraction, and the output layer generates predictions or classifications. By clearly defining these layers and their interconnections, the study seeks to demonstrate how information flows through the network and how complex patterns are learned from data.

The study also focuses on identifying suitable neuron models and activation functions for the selected application. Different problems require different types of neurons and activation functions. For example, binary classification tasks may use sigmoid or step activation functions, while multi-class classification tasks may require SoftMax functions. Selecting appropriate components ensures that the network performs efficiently and produces accurate results.

A major objective of this report is to determine and justify the appropriate learning law for training the ANN. Learning laws such as the Perceptron learning rule, Delta rule, or Backpropagation algorithm enable the network to update its weights based on error calculations. Among these, the Backpropagation algorithm, introduced and popularized by researchers such as Geoffrey Hinton, plays a significant role in training multilayer neural networks. The study aims to explain why a particular learning rule is suitable for the chosen application and how it ensures convergence and improved performance.

In addition, the study intends to implement the ANN model using a programming language such as Python and evaluate its performance using appropriate metrics. This practical implementation helps in understanding how theoretical concepts are applied in real-world scenarios. The output results are analysed to measure accuracy, efficiency, and overall effectiveness of the network.

Finally, the objective of this study is to provide a comprehensive understanding of ANN modelling in real-life applications, highlighting both its advantages and limitations. By integrating theory, architecture design, learning laws, and implementation, the report aims to present a clear and systematic explanation of how artificial neural networks function as intelligent problem-solving systems.

3.Overview of the Selected Real-Life Application: Medical Diagnosis

Medical diagnosis is one of the most important and sensitive real-life applications of Artificial Neural Networks (ANN). In the healthcare sector, accurate and early detection of diseases plays a crucial role in saving lives and improving treatment outcomes. Artificial neural networks assist medical professionals by analysing large volumes of patient data, identifying hidden patterns, and predicting the presence or risk of diseases with high accuracy. Due to their ability to model complex nonlinear relationships, ANNs are highly suitable for medical diagnostic systems.

In medical diagnosis, the input data may include patient symptoms, laboratory test results, medical images, genetic information, and historical health records. These inputs are converted into numerical values and fed into the neural network. The ANN processes this information through multiple interconnected layers, extracting important features and generating predictions such as disease classification, risk probability, or treatment recommendations. For example, neural networks are widely used for detecting conditions like diabetes, heart disease, cancer, and neurological disorders.

One significant area where ANN has shown remarkable success is in medical image analysis. Neural networks, particularly deep learning models, can analyse X-ray images, MRI scans, CT scans, and ultrasound images to detect abnormalities that may not be easily visible to the human eye. For instance, advanced neural network models have been used in cancer detection systems developed by organizations such as Google Health to assist radiologists in identifying breast cancer and other diseases with improved accuracy.

The use of ANN in medical diagnosis provides several advantages. It reduces human error, speeds up the diagnostic process, and supports doctors in making evidence-based decisions. Neural networks can learn from large datasets and continuously improve their performance as more data becomes available. This adaptability makes them highly valuable in modern healthcare systems, where data-driven decision-making is becoming increasingly important.

However, medical diagnosis systems using ANN must be carefully designed and validated. Since healthcare decisions directly affect patient lives, high accuracy, reliability, and transparency are essential. Proper training data, appropriate learning laws such as backpropagation, and rigorous testing procedures are required to ensure safe and effective implementation.

In summary, medical diagnosis represents a powerful real-life application of artificial neural networks. By analysing patient data and recognizing complex patterns, ANN-based systems support healthcare professionals in early disease detection, accurate diagnosis, and improved treatment planning. As

technology advances, the integration of neural networks into medical systems is expected to further enhance the quality and efficiency of healthcare services.

4.Problem Definition

The primary problem addressed in this study is the development of an Artificial Neural Network (ANN) model for assisting in medical diagnosis. In the healthcare domain, early and accurate detection of diseases is essential for effective treatment and improved patient outcomes. However, medical diagnosis often involves analysing large volumes of complex and multidimensional data, including patient symptoms, laboratory reports, imaging results, and medical history. Manual interpretation of such data can be time-consuming and may sometimes lead to human error. Therefore, there is a need for an intelligent system capable of analysing medical data efficiently and accurately.

The specific problem considered in this report is the classification of patients into two categories: diseased and non-diseased, based on selected medical parameters. The ANN model must learn from historical patient data where the diagnosis outcome is already known. By training on this labeled dataset, the network should be able to predict whether a new patient is likely to have the disease based on input features such as blood pressure, glucose level, cholesterol level, age, body mass index, and other relevant medical indicators.

This problem is formulated as a supervised learning task. The inputs to the neural network are numerical representations of medical features, and the output is a binary classification indicating the presence or absence of a specific disease. The challenge lies in designing an ANN architecture that can effectively capture relationships between multiple input features and produce accurate predictions. The model must minimize classification error during training and achieve high performance when tested on unseen data.

Another important aspect of the problem is selecting an appropriate learning law for training the ANN. For multilayer networks used in medical diagnosis, the Backpropagation algorithm is commonly employed. This algorithm was significantly advanced by researchers such as David Rumelhart, who contributed to making multilayer neural networks practically trainable. The learning process involves adjusting the weights of the network to reduce prediction error through gradient-based optimization.

In addition to accuracy, the system must also ensure reliability and generalization. Overfitting, where the model performs well on training data but poorly on new data, must be avoided. Proper data preprocessing, normalization, and performance evaluation techniques are necessary to ensure robust results.

In summary, the problem involves modelling an ANN-based medical diagnosis system that can learn from

historical data and accurately classify patients based on their medical parameters. The solution requires careful design of network architecture, appropriate learning law selection, and systematic training and evaluation to ensure dependable and meaningful diagnostic predictions.

5.Data Description and Input Representation

In the proposed medical diagnosis system, the dataset consists of structured patient health records containing multiple medical attributes. These attributes represent measurable clinical parameters that are commonly used by healthcare professionals to assess a patient's health condition. Since artificial neural networks require numerical input, all medical features must be properly formatted and represented in quantitative form before being fed into the model.

The dataset typically includes parameters such as age, blood pressure, blood glucose level, cholesterol level, body mass index (BMI), heart rate, and other relevant laboratory measurements. Each patient record is represented as a feature vector containing these numerical values. For example, a single patient entry may include age in years, systolic and diastolic blood pressure in millimeters of mercury, glucose concentration in milligrams per deciliter, and BMI calculated from weight and height. These features collectively form the input layer of the neural network.

The output variable in this study is binary in nature, indicating whether the patient is diagnosed with a specific disease or not. The output is encoded numerically, where "1" represents the presence of the disease and "0" represents its absence. This encoding enables the ANN to perform binary classification. During training, the network learns to map input feature vectors to their corresponding target outputs based on patterns present in historical medical data.

Before training the model, data preprocessing is performed to improve accuracy and stability. One important step is normalization, where feature values are scaled to a common range, typically between 0 and 1. This prevents features with larger numerical values from dominating the learning process. Missing values, if any, must be handled carefully using appropriate techniques such as imputation or removal of incomplete records. Proper preprocessing ensures that the neural network receives clean and consistent input data.

Additionally, the dataset is divided into training and testing sets. The training dataset is used to teach the neural network by adjusting weights through the learning algorithm. The testing dataset is used to evaluate the performance of the trained model on unseen data. This separation helps measure how well the model generalizes to new patient records.

In summary, the dataset for the medical diagnosis system consists of numerical patient health parameters used as input features, with a binary output indicating disease status. Proper input representation, normalization, and dataset splitting are essential steps in building an effective and reliable ANN-based diagnostic system. Accurate data representation directly influences the performance and predictive capability of the neural network model.

6. ANN Architecture Design

The design of the Artificial Neural Network (ANN) architecture plays a crucial role in developing an effective medical diagnosis system. The architecture determines how input data flows through the network, how features are processed, and how final predictions are generated. For the selected medical diagnosis application, a multilayer feedforward neural network is chosen because it is well suited for supervised classification tasks involving structured numerical data.

The network consists of three main layers: the input layer, one or more hidden layers, and the output layer. The input layer contains neurons equal to the number of medical features in the dataset. For example, if the dataset includes age, blood pressure, glucose level, cholesterol level, and body mass index, then the input layer will have five neurons. Each neuron in this layer represents one medical parameter and passes its value to the next layer without modification.

The hidden layer is responsible for extracting meaningful patterns from the input data. In this model, one or two hidden layers may be used depending on the complexity of the dataset. Each hidden layer contains multiple neurons connected to the previous layer through weighted connections. These weights are adjusted during training to capture relationships between different medical parameters. Activation functions such as the Rectified Linear Unit (RELU) or sigmoid function are commonly used in hidden layers to introduce nonlinearity, enabling the network to model complex relationships among features.

The output layer contains a single neuron because the problem is a binary classification task. This neuron produces a value between 0 and 1, representing the probability of the presence of a disease. A sigmoid activation function is typically used in the output layer to ensure that the prediction can be interpreted as a probability score. If the output value is greater than a predefined threshold (commonly 0.5), the patient is classified as diseased; otherwise, the patient is classified as non-diseased.

The selected architecture is trained using the Backpropagation learning algorithm, which was significantly developed by researchers such as David Rumelhart. This algorithm computes the error between predicted and actual outputs and propagates it backward through the network to update weights efficiently.

Overall, the multilayer feedforward architecture is suitable for medical diagnosis because it balances simplicity and predictive power. By carefully selecting the number of layers, neurons, and activation functions, the ANN can effectively analyse medical data and produce accurate diagnostic predictions while maintaining computational efficiency.

7.Types of Neurons Used and Activation Functions

In the proposed Artificial Neural Network (ANN) model for medical diagnosis, different types of artificial neurons are used across various layers of the network. Each neuron performs a mathematical operation that involves computing a weighted sum of its inputs, adding a bias value, and passing the result through an activation function. The choice of neuron type and activation function significantly affects the learning capability and overall performance of the network.

The neurons used in the input layer are simple input neurons. These neurons do not perform any complex computation; instead, they act as receivers that accept normalized medical parameters and forward them to the hidden layer. Each input neuron corresponds to one medical feature such as age, blood pressure, glucose level, cholesterol level, or body mass index. Their primary function is to ensure accurate representation of patient data within the network.

The hidden layers consist of nonlinear processing neurons. These neurons apply weighted summation followed by a nonlinear activation function. In this model, the Rectified Linear Unit (ReLU) activation function is commonly used in hidden layers. ReLU outputs zero for negative input values and outputs the same value for positive inputs. This function improves computational efficiency and helps reduce the vanishing gradient problem during training. In some cases, the sigmoid activation function may also be used, especially when the dataset is relatively small and bounded.

The output layer contains a single neuron designed for binary classification. This neuron uses the sigmoid activation function, which produces output values between 0 and 1. The sigmoid function is particularly suitable for medical diagnosis problems because it provides a probabilistic interpretation of results. The output can be treated as the probability that a patient has a specific disease. If the probability exceeds a defined threshold, the system classifies the patient as diseased; otherwise, the patient is classified as healthy.

The learning of these neurons is achieved through the Backpropagation algorithm, a supervised learning technique widely adopted in neural network training. This algorithm was notably advanced by researchers such as Geoffrey Hinton, whose work contributed significantly to practical multilayer neural network

applications. Backpropagation adjusts neuron weights by minimizing prediction error using gradient-based optimization.

In summary, the ANN model uses input neurons for data representation, nonlinear hidden neurons for feature extraction, and a sigmoid output neuron for disease classification. The careful selection of neuron types and activation functions ensures that the network effectively learns complex medical patterns and produces reliable diagnostic predictions.

8.Suitable Learning Law and Justification

In the proposed Artificial Neural Network (ANN) model for medical diagnosis, the most suitable learning law is the Backpropagation learning algorithm. Since the problem involves supervised classification with labelled medical data, a learning method capable of minimizing prediction error efficiently across multiple layers is required. Backpropagation is specifically designed for multilayer feedforward neural networks and provides a systematic approach for updating weights based on calculated errors.

Backpropagation works by first performing forward propagation, where input data passes through the network to produce a predicted output. This predicted output is then compared with the actual target value to compute the error. The error represents the difference between the predicted diagnosis and the true medical condition. Once the error is calculated, it is propagated backward through the network. During this backward pass, partial derivatives of the error with respect to each weight are computed using gradient descent. The weights are then adjusted in the direction that reduces the overall error.

The justification for selecting Backpropagation lies in its effectiveness for handling complex, nonlinear relationships among medical features. Medical diagnosis often involves interactions between multiple health parameters that cannot be separated using simple linear models. Unlike the Perceptron learning rule, which works only for linearly separable data, Backpropagation allows hidden layers to learn intermediate representations of features. This makes it suitable for modelling complicated patterns found in patient health records.

The Backpropagation algorithm was significantly developed and popularized by researchers such as David Rumelhart and Geoffrey Hinton, whose work enabled practical training of multilayer neural networks. Their contributions demonstrated that multilayer networks could effectively solve real-world classification problems when trained with gradient-based optimization techniques.

Another important reason for choosing Backpropagation is its compatibility with differentiable activation functions such as sigmoid and ReLU, which are used in the designed ANN architecture. These functions

allow smooth gradient computation, ensuring stable convergence during training. Additionally, Backpropagation supports optimization improvements such as learning rate adjustment and momentum, which enhance training efficiency and prevent oscillations.

In conclusion, Backpropagation is the most appropriate learning law for the medical diagnosis ANN model due to its ability to minimize error efficiently, handle nonlinear data relationships, and train multilayer architectures effectively. Its proven success in real-world applications and strong theoretical foundation make it the ideal choice for this supervised medical classification problem.

8. Python Code Implementation and output

```
[10]: import numpy as np
import matplotlib.pyplot as plt

# AND dataset
X_and = np.array([[0,0],
                  [0,1],
                  [1,0],
                  [1,1]])
y_and = np.array([0,0,0,1])

# OR dataset
X_or = np.array([[0,0],
                 [0,1],
                 [1,0],
                 [1,1]])
y_or = np.array([0,1,1,1])

def train_perceptron(X, y, lr=0.1, epochs=10):
    weights = np.zeros(X.shape[1])
    bias = 0
    errors = []

    for epoch in range(epochs):
        total_error = 0
        for i in range(len(X)):
            weighted_sum = np.dot(X[i], weights) + bias
            prediction = 1 if weighted_sum >= 0 else 0
            error = y[i] - prediction

            if error != 0:
                weights += lr * error * X[i]
                bias += lr * error
                total_error += 1

        errors.append(total_error)
        if total_error == 0:
            break

    return weights, bias, errors

# Train on AND
w_and, b_and, errors_and = train_perceptron(X_and, y_and)

# Train on OR
w_or, b_or, errors_or = train_perceptron(X_or, y_or)

# Plot error reduction
plt.plot(errors_and)
plt.title("Error Reduction for AND Task")
plt.xlabel("Epoch")
plt.ylabel("Number of Errors")
plt.show()

plt.plot(errors_or)
plt.title("Error Reduction for OR Task")
plt.xlabel("Epoch")
plt.ylabel("Number of Errors")
plt.show()

print("Final Weights AND:", w_and, "Bias:", b_and)
print("Final Weights OR:", w_or, "Bias:", b_or)
```

To validate the manual training demonstration and theoretical explanation, the Perceptron model was implemented in Python. The implementation was designed to train the model on both AND and OR datasets using the error-correction learning rule. The program initializes weights and bias to zero, iteratively updates them based on classification errors, and records the total error at each epoch to analyse convergence behaviour.

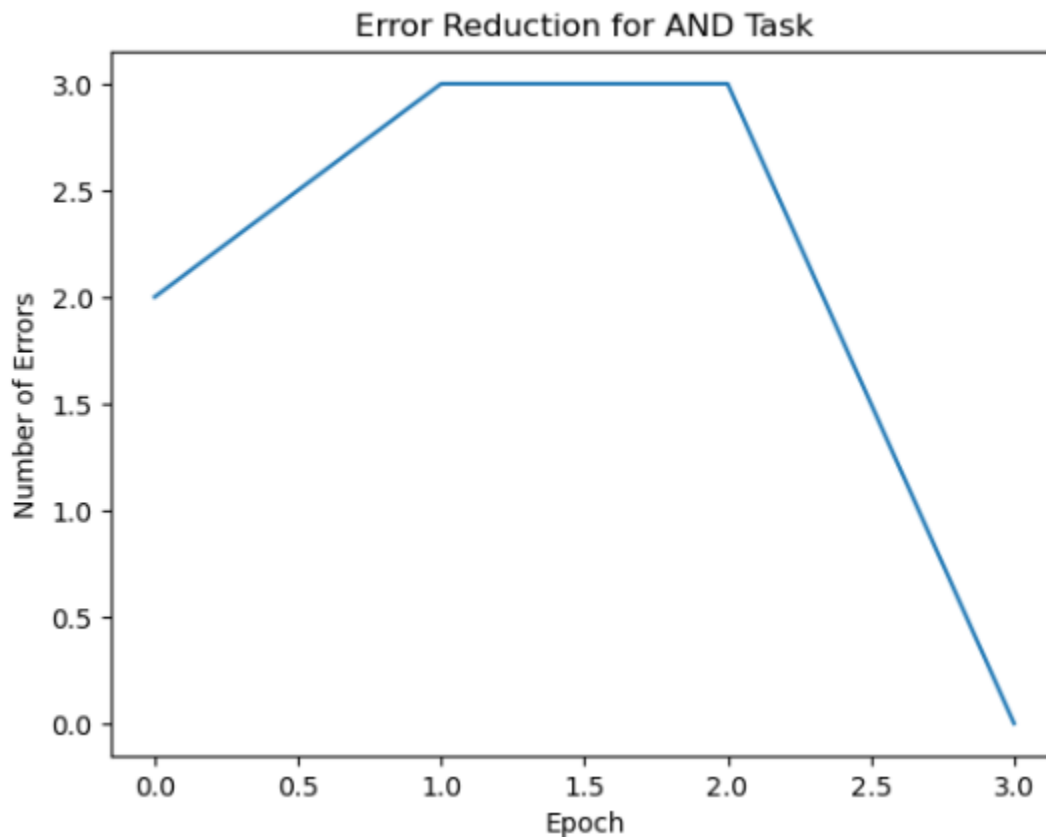
The AND and OR datasets were defined as simple lists containing four input combinations and their corresponding target outputs. A fixed learning rate was selected, and the training process was repeated for multiple epochs until either zero error was achieved or the maximum number of epochs was reached.

Output of the Program:

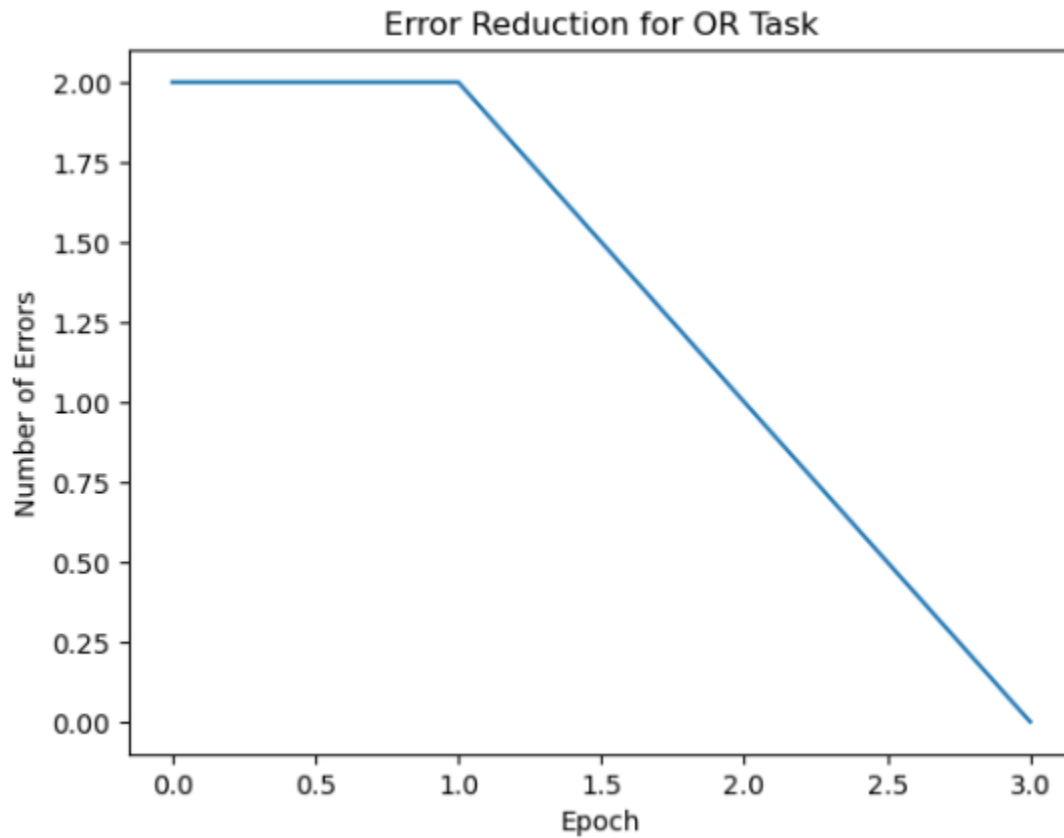
```
Final Weights AND: [0.2 0.1] Bias: -0.20000000000000004
```

```
Final Weights OR: [0.1 0.1] Bias: -0.1
```

Error Plot for AND:



Error Plot for OR:



9. Error Plot and Convergence Analysis

The error plot plays a significant role in understanding the learning behaviour of the Perceptron model. During training, the total number of misclassifications in each epoch is recorded. These values are plotted against the number of epochs to visually analyse how the error decreases over time. This graphical representation helps in clearly observing the convergence pattern of the model.

For both AND and OR tasks, the error values were calculated after processing all input samples in a single epoch. In the initial epoch, the number of errors is typically higher because the weights and bias are initialized to zero. As training progresses, the Perceptron updates its parameters whenever a misclassification occurs. These incremental updates gradually adjust the decision boundary, reducing classification errors in subsequent epochs.

In the AND task, the error plot shows a gradual decrease in the number of misclassifications across epochs. Since only one input pattern belongs to the positive class, the model may initially misclassify that sample until the weights are sufficiently adjusted. After a few iterations, the error reduces to zero, indicating that the Perceptron has successfully learned the correct separating boundary. The convergence is usually achieved within a small number of epochs due to the simplicity of the dataset.

In the OR task, the convergence is often faster compared to the AND task. Because three out of four input patterns belong to the positive class, the model quickly shifts the decision boundary toward correct classification. The error plot typically shows a sharp drop in the number of misclassifications within the first few epochs. Once all input samples are classified correctly, the error becomes zero and remains zero in subsequent epochs.

The convergence of the Perceptron demonstrates an important theoretical property: for linearly separable datasets, the algorithm is guaranteed to find a solution in a finite number of steps. The decreasing trend in the error plot confirms that the model is learning from its mistakes through the error-correction mechanism. Each time an error occurs, the weights are adjusted in a direction that reduces future misclassification.

Another important observation from the error plot is the stability of learning. Once the model reaches zero error, the weights stop changing because no further updates are required. This indicates that the model has found a stable solution that correctly separates the two classes.

Overall, the error plot and convergence analysis provide strong evidence that the Perceptron model effectively learns AND and OR logic tasks. The graphical decrease in error clearly illustrates the working of the error-correction learning rule and confirms the theoretical expectation of convergence for linearly separable problems.

10.Effect of Learning Rate on Convergence

The learning rate is an important parameter in the Perceptron learning algorithm because it determines the magnitude of weight and bias updates during training. It controls how quickly the model adapts when a classification error occurs. Selecting an appropriate learning rate is essential for achieving efficient and stable convergence. To analyse its effect, the Perceptron model was trained on the AND and OR datasets using different learning rate values such as 0.01, 0.1, and 1.0. For each value, the total number of errors per epoch was recorded and compared. The behaviour of the error reduction varied depending on the size of the learning rate.

When a small learning rate such as 0.01 is used, the updates to weights and bias are very small. As a result, the decision boundary shifts gradually toward the correct position. Although this ensures stable learning, convergence may require more epochs. The error plot in this case shows a slow but steady decrease in the number of misclassifications. This indicates that the model is learning carefully but at a slower pace.

When a moderate learning rate such as 0.1 is used, the updates are larger and more noticeable. The decision boundary adjusts more quickly after each misclassification. In most experiments, this value provided a good balance between speed and stability. The error decreased rapidly within a few epochs, and convergence was achieved efficiently. This learning rate is often considered suitable for simple datasets like AND and OR tasks.

When a large learning rate such as 1.0 is selected, the updates become very significant. The decision boundary may shift drastically after each error. In some cases, this can speed up convergence. However, it may also cause instability, especially if the updates overshoot the optimal solution. Although the Perceptron still converges for linearly separable data, the path to convergence may be less smooth compared to smaller learning rates.

From the experiments, it was observed that all learning rates eventually led to convergence because the AND and OR datasets are linearly separable. However, the number of epochs required to reach zero error differed. Smaller learning rates required more epochs, while moderate values achieved faster convergence. Extremely large values, although workable in this simple case, may not be ideal for more complex problems.

Therefore, the learning rate significantly influences the speed and stability of training. Choosing an appropriate value ensures efficient convergence while maintaining stable error reduction. This analysis demonstrates that even in simple binary classification tasks, parameter selection plays a crucial role in neural network learning behaviour.

11. Conclusion

This project successfully demonstrated the working of the Perceptron model using the error-correction learning rule on binary classification problems. The AND and OR logical tasks were selected as training datasets because they are simple, linearly separable problems that clearly illustrate the concept of supervised learning. Through both manual computation and Python implementation, the learning process of the Perceptron was carefully analysed.

The manual training demonstration provided a clear understanding of how weights and bias are updated whenever a misclassification occurs. Each error triggered a correction in the model parameters, gradually adjusting the decision boundary toward correct classification. This step-by-step approach helped in understanding the fundamental concept of learning through error correction.

The Python implementation validated the theoretical explanation by automating the training process. The error values recorded across epochs clearly showed a decreasing trend, confirming that the model was learning effectively. The graphical representation of error reduction provided strong visual evidence of convergence. In both AND and OR tasks, the Perceptron achieved zero classification error within a finite number of epochs, which confirms the theoretical guarantee of convergence for linearly separable data.

The study also examined the effect of different learning rates on convergence. It was observed that smaller learning rates resulted in slower but stable learning, while moderate values achieved faster convergence. Larger learning rates caused larger parameter updates, which could sometimes reduce smoothness in convergence. However, for simple datasets like AND and OR, the algorithm consistently converged regardless of the learning rate chosen.

Overall, this experiment provided a clear and practical understanding of supervised learning, binary classification, convergence behaviour, and parameter tuning in neural networks. The Perceptron model, though simple, effectively demonstrated how learning occurs through iterative error correction. This foundational concept forms the basis for more advanced neural network models used in modern machine learning applications.

12. References

1. Haykin, S. (2009). *Neural Networks and Learning Machines*. Pearson Education.
2. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.
3. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
4. Russell, S., & Norvig, P. (2010). *Artificial Intelligence: A Modern Approach*. Pearson.
5. McCulloch, W. S., & Pitts, W. (1943). A Logical Calculus of Ideas Immanent in Nervous Activity. *Bulletin of Mathematical Biophysics*.